

ADT Sorted Multi-map

Seminar 4 DSA - completed.

22.04.2026

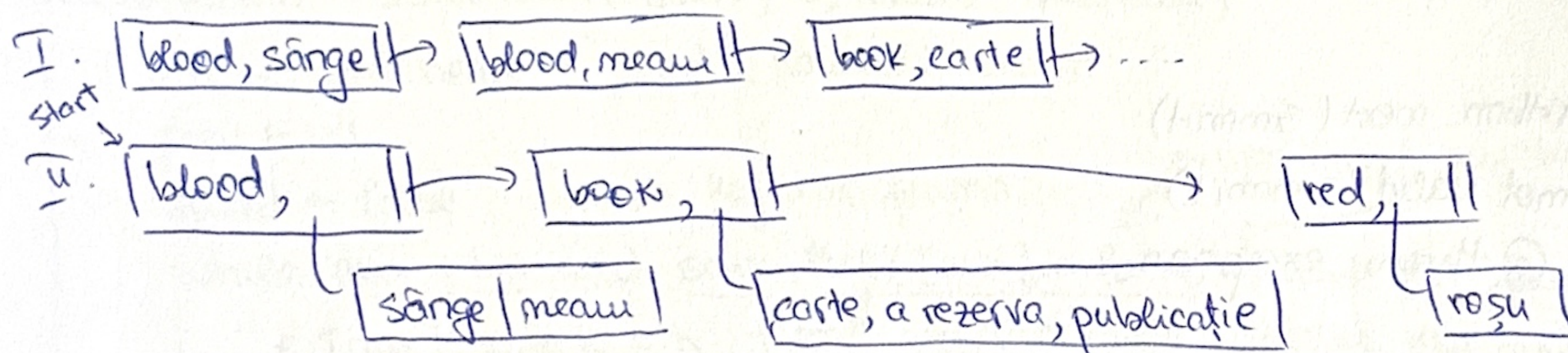
- Map: has key-value pairs
each key is unique
each key has one value associated to it.
- Multi-map: a key can have multiple values associated to it.
- Sorted multi-map: has a relation R (keys)
we don't order based on values.

Implement ADT SMH - use a SLL with dynamic allocation.

book: - carte, a rezerva, publicatie

red: - rosu

blood: - sange, meam



DS level: - DA,
- SLL, DLL

ADT level: - Bag
- List

Representation:
(structures)

Value:	position	iterator
	getPosition	getCurrent
list=DA	$\Theta(1)$	$\Theta(1)$
list=LL	$O(n)$	$\downarrow \Theta(1)$

$\Theta(1)$ because no traversal is needed, the iterator stores a POINTER to the current node/value

<u>TElem</u> K: TKey vL: List
<u>SMHNode</u> info: <u>TElem</u> next: <u>↑SMHNode</u>
<u>SMH</u> head: <u>↑SMHNode</u> size: Integer rel: Relation
<u>SMHIterator</u> cNode: <u>↑SMHNode</u> cValue: ListIterator smm: SMH

Implementation:

init
valid
next
getCurrent

first → resets the iterator to the beginning, we will do that in init(), so we don't need first()
→ discussion.

current node from SMH (ref.)
iterator for the list of values associated with cNode

①

Printing using iterator over SHM: (always the same)

Subalgorithm $\text{print}(\text{smmm})$ is:

$\Theta(1)$ $\text{iterator}(\text{smmm}, \text{it})$

while $\text{valid}(\text{it})$ execute: $\Theta(1)$

- $\text{getCurrent}(\text{it}, \langle k, v \rangle)$
- @ print k and v
- $\text{next}(\text{it})$

Operations for iterator:

Subalgorithm $\text{init}(\text{smm}, \text{smmmit})$ is:

$\Theta(1)$ $\text{smmmit.smm} \leftarrow \text{smm}$

$\text{smmmit.cNode} \leftarrow \text{smm.head}$

if $\text{smm.head} \neq \text{NIL}$ then // exists a node to be iterated

- $\text{iterator}([\text{smm.head}].\text{info.vL}, \text{smmmit.cValue})$

Subalgorithm $\text{next}(\text{smmmit})$:

$\Theta(1)$ if not $\text{valid}(\text{smmmit})$

- @ throw exception

$\text{next}(\text{smmmit.cValue})$

if not $\text{valid}(\text{smmmit.cValue})$ then // values for this key are done

- $\text{smmmit.cNode} \leftarrow [\text{smmmit.cNode}].\text{next}$ // next key node

if $\text{smmmit.cNode} \neq \text{NIL}$ then // if another node exists

- $\text{iterator}([\text{smmmit.cNode}].\text{info.vL}, \text{smmmit.cValue})$

function $\text{valid}(\text{smmmit})$:

$\Theta(1)$ if $\text{smmmit.cNode} = \text{NIL}$ then ; current node points to NIL?

- $\text{valid} \leftarrow \text{False}$

else

- $\text{valid} \leftarrow \text{True}$

function $\text{getCurrent}(\text{smmmit})$:

$\Theta(1)$ if not $\text{valid}(\text{smmmit})$ then

- @ throw exception

$k \leftarrow [\text{smmmit.cNode}].\text{info.k}$

$v \leftarrow \text{getCurrent}(\text{smmmit.cValue})$

$\text{getCurrent} \leftarrow \langle k, v \rangle$

ListIterator (taken for granted)

add K, N pair

- find node with K ...

remove K, N

- find node w K ...

search K

- find node w K.

// K = book, KNode = "book", prevNode = "blood"

// K = blood, KNode = "blood", prevNode = Nil (this is the first node)

// K = day, KNode = Nil, prevNode = book (last smaller existing key)

// K = air, KNode = Nil, prevNode = Nil (no smaller existing key than air)

Subalgorithm searchNode(smm, K, KNode, prevNode)

aux ← smm.head ; start "traversal" point

prev ← Nil

found ← false ; assume key not found

current key ≤ searched key

while aux ≠ Nil and smm.key([aux].info.K, K) and not found ex:

if [aux].info.K = K then ; current node matches key we want

found ← True

else

prev ← aux

aux ← [aux].next

if found then:

KNode ← aux

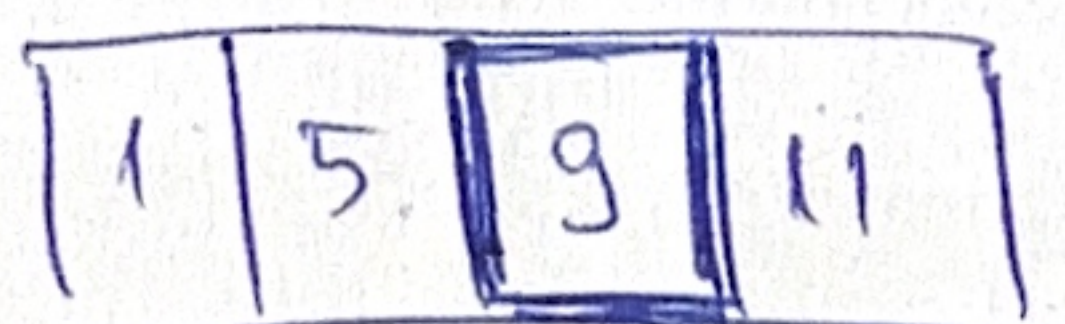
prevNode ← prev

else // aux is either past the correct position or at a larger key

KNode ← Nil

prevNode ← prev ; // because aux passed the insertion position.

K = 10



"should 1 be in FRONT of 10?"

"should 5 be in FRONT of 5?"

"should 11 be in FRONT of 10?" NO ⇒ STOP ⇒ prev.

Subalgorithm add (smm, k, v):

searchNode (smm, k, KNode, prevNode) $\leftarrow O(n)$ $\rightarrow$ if $\exists k \Rightarrow$ $\begin{cases} \text{prevNode} = \text{prev} \\ \text{KNode} = \text{aux} \end{cases}$
 if KNode $\neq$ Nil then // not in the map so we need to add it $\rightarrow$ if $\nexists k \Rightarrow$ $\begin{cases} \text{KNode} = \text{Nil} \\ \text{prevNode} = \text{prev} \end{cases}$
 addToEnd ([KNode].info.vL, v) $\leftarrow O(1)$ (at the end bc sorted)

else // key already exists

newNode $\leftarrow$ allocate(); create new node in memory
 [newNode].info.k $\leftarrow$ k; set the key

init ([newNode].info.vL); initialize empty value list.

addToEnd ([newNode].info.vL, v) $\leftarrow O(1)$

if prevNode = Nil then // add it as the first elem

[newNode].next $\leftarrow$ smm.head

smm.head $\leftarrow$ newNode

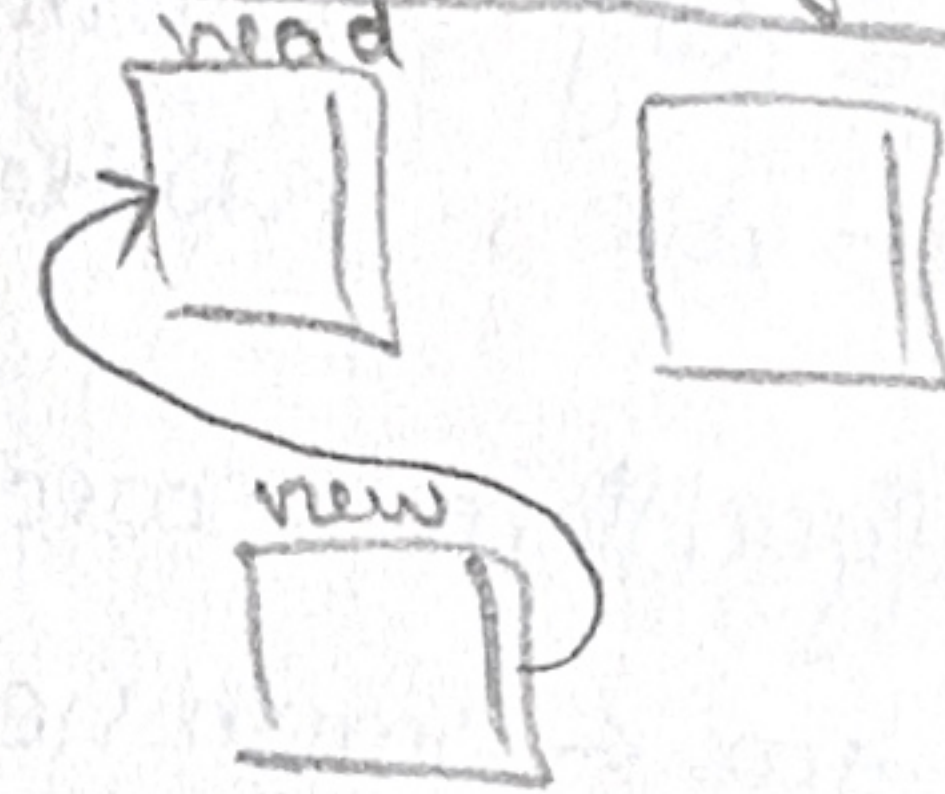
else // add it as normal (after previous)

[newNode].next $\leftarrow$ [prevNode].next

[prevNode].next $\leftarrow$ newNode

smm.size $\leftarrow$ smm.size + 1.

at beginning:



at "middle":

